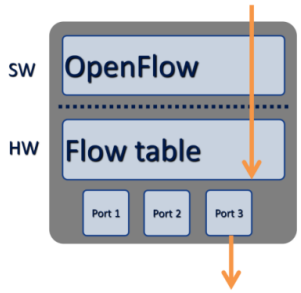
Advanced Communication Architectures (TSM_AdvComArc)

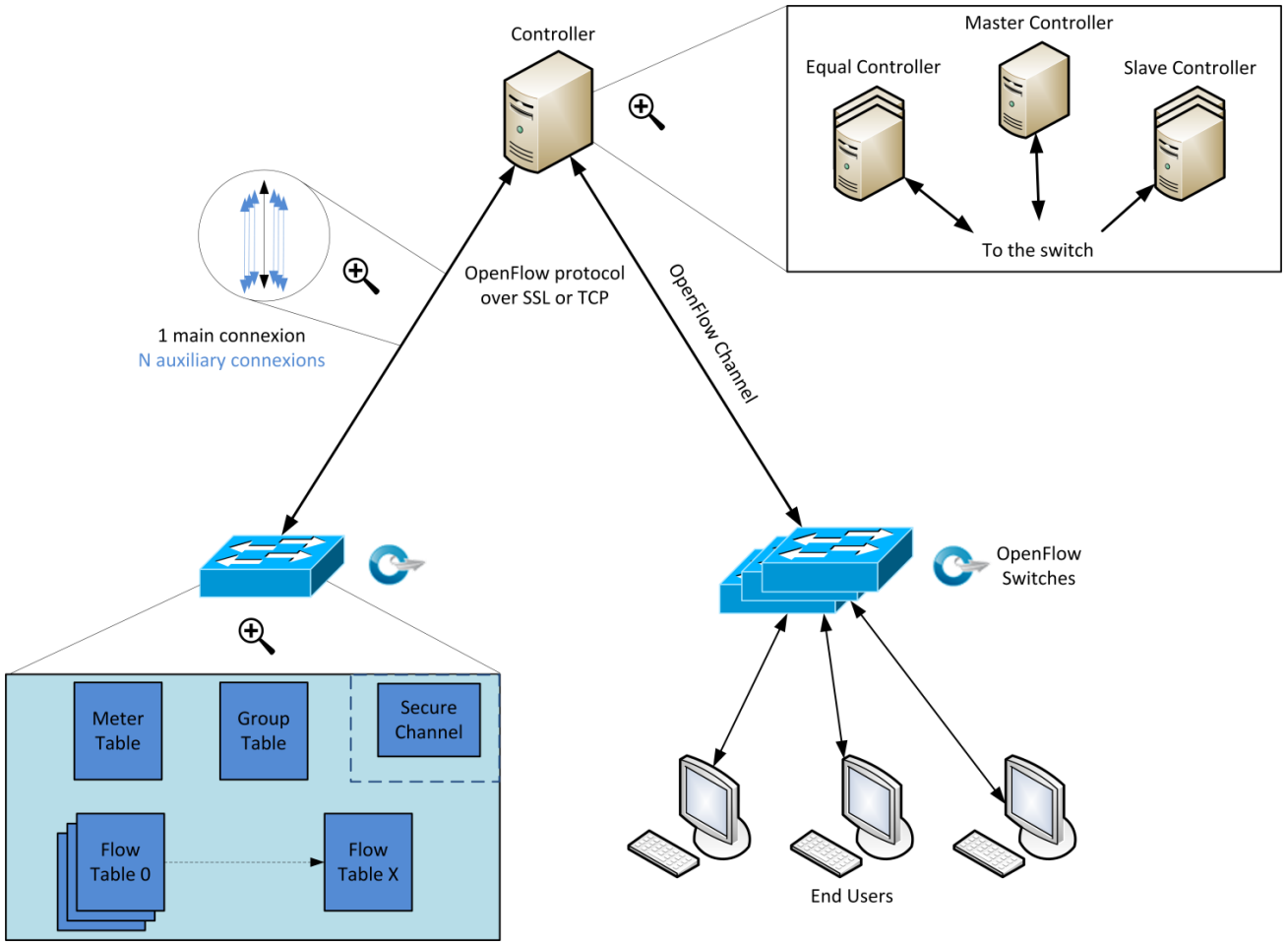
OpenFlow

OpenFlow – Definition

- OpenFlow is a communication protocol offering an interface between control and forwarding layer of a SDN architecture (Southbound API)
- OpenFlow is based on concept of flow; a flow is a sequence of packets sharing a common header field's value
- OpenFlow allows direct access and manipulation of the forwarding plane of network devices
- OpenFlow is standardized by ONF
  - ONF (Open Networking Fondation) is a nonprofit organization created in march 2011 by Deutsche Telecom, Facebook, Google, Verizon, Microsoft and Yahoo!
- Current version OpenFlow specification is 1.5.1 (2015)



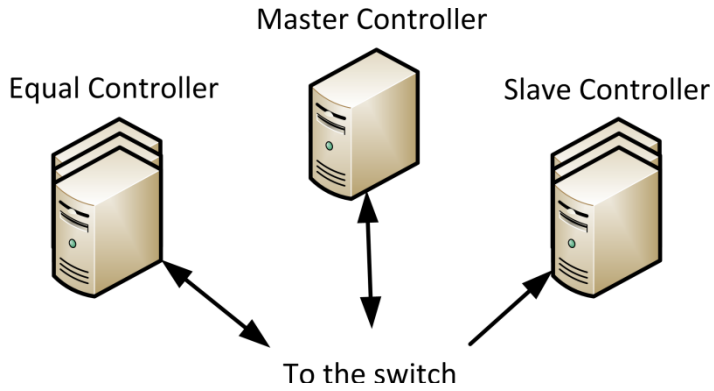
OpenFlow – Architecture



OpenFlow – Components – 1

Controller

- Communicates with the switches thanks to the OpenFlow protocol
- Manages flow rules in devices
- Offers North API used by applications like:
  - Firewall, IDS, routing, switching
- 3 possible roles:
  - 1 Master: read-write access / unique
  - N Equal (default): read-write access
  - N Slaves: read only access
- Multiple controller can be used
  - Switch maintain connection with each one
  - Controllers coordinates switch management amongst themselves



OpenFlow – Components – 2a

- **Examples of OpenFlow controller**

Controller	Programming Language	OpenFlow Versions Supported	Primary Use Case	Scalability	Modularity	Notes
Ryu	Python	1.0 – 1.5	Research, prototyping, light use	Medium	High	Simple API, good docs
POX	Python	1	Education, quick prototyping	Low	Medium	No longer actively developed
Floodlight	Java	1	Enterprise, data centers	Medium	High	Extensible via modules
ONOS	Java	1.0 – 1.5 (plus others)	Carrier-grade networks	High	High	Supports HA and clustering
OpenDaylight	Java	1.0 – 1.5 (plus others)	Enterprise, telco, data centers	High	Very High	Large ecosystem, steep learning curve
Trema	C, Ruby	1	Lightweight apps, research	Low	Low	Fast but limited community
Beacon	Java	1	High-performance apps	Medium	Medium	Basis for Floodlight

OpenFlow – Components – 2b

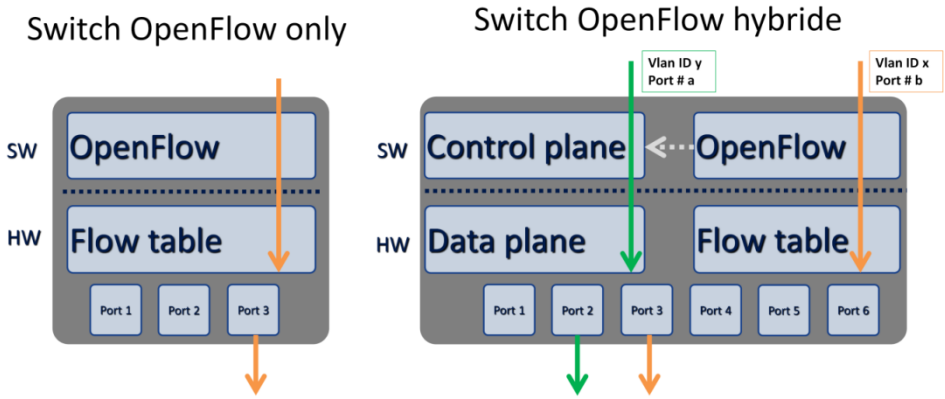
- **Examples of OpenFlow controller**

Controller	License Type	Price	Notes
Ryu	Apache 2.0 (Open Source)	**Free**	Community-supported
POX	Apache 2.0 (Open Source)	**Free**	No longer actively maintained
Floodlight	Apache 2.0 (Open Source)	**Free**	Supported by the Big Switch community
ONOS	Apache 2.0 (Open Source)	**Free**	Maintained by the Open Networking Foundation (ONF)
OpenDaylight	Eclipse Public License	**Free**	Enterprise-grade but open source
Trema	GPLv2 (Open Source)	**Free**	Less popular, community-driven
Beacon	GPLv2 (Open Source)	**Free**	No longer actively maintained

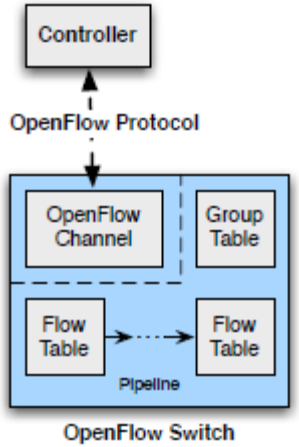
OpenFlow – Components – 3

Switch

- 2 types of switch
  - OpenFlow only
  - OpenFlow hybrides



- Multiple tables
  - Flow table(s)
  - Meter table
  - Group table



OpenFlow – Components – 4

Examples of switches supporting OpenFlow

- Software switch
  - Open vSwitch
- Hardware switch
  - HP ProCurve
  - NEC IP8800
  - HP 3800 Switch Series
  - Cisco Nexus 3000
  - Cisco Catalyst 3850
  - …

OpenFlow – Flow Table – 1

- A flow table consists of flow entries :

Match Fields	Priority	Counters	Instructions	Timeouts	Cookie

  - **match fields**: to match against packets. These consist of the ingress port and packet headers, and optionally metadata specified by a previous table
  - **priority**: matching precedence of the flow entry
  - **counters**: updated when packets are matched
  - **instructions**: to modify the action set or pipeline processing
  - **timeouts**: maximum amount of time or idle time before flow entry is expired by the switch
  - **cookie**: opaque data value chosen by the controller. May be used by the controller to filter flow statistics, flow modifications and flow deletion. Not used when processing packets

OpenFlow – Flow Table – 2

- At least one flow table but possibly more
- The match fields and priority taken together identify a unique flow entry in the flow table
- Table matching stops after the first match
- The flow entry that wildcards all fields and has priority equal to 0 is called “table-miss flow entry”. This entry is executed in case of unmatching
- By default, if no table-miss entry exists, unmatched packets are dropped

OpenFlow – Flow Table – 2

Field		Description
OXM_OF_IN_PORT	*Required*	Ingress port. This may be a physical or switch-defined logical port.
OXM_OF_ETH_DST	*Required*	Ethernet destination address. Can use arbitrary bitmask
OXM_OF_ETH_SRC	*Required*	Ethernet source address. Can use arbitrary bitmask
OXM_OF_ETH_TYPE	*Required*	Ethernet type of the OpenFlow packet payload, after VLAN tags.
OXM_OF_IP_PROTO	*Required*	IPv4 or IPv6 protocol number
OXM_OF_IPV4_SRC	*Required*	IPv4 source address. Can use subnet mask or arbitrary bitmask
OXM_OF_IPV4_DST	*Required*	IPv4 destination address. Can use subnet mask or arbitrary bitmask
OXM_OF_IPV6_SRC	*Required*	IPv6 source address. Can use subnet mask or arbitrary bitmask
OXM_OF_IPV6_DST	*Required*	IPv6 destination address. Can use subnet mask or arbitrary bitmask
OXM_OF_TCP_SRC	*Required*	TCP source port
OXM_OF_TCP_DST	*Required*	TCP destination port
OXM_OF_UDP_SRC	*Required*	UDP source port
OXM_OF_UDP_DST	*Required*	UDP destination port

OpenFlow – Flow Table – 3

- Examples of flow table entries :

Switching

MAC src	MAC dst	IP Src	IP Dst	Vlan	TCP src	TCP dst	Instructions
*	0000-0C01-0203	*	*	*	*	*	Ouput Port 1
*	0302-0100-0000	*	*	*	*	*	Ouput Port 2

Routing

MAC src	MAC dst	IP Src	IP Dst	Vlan	TCP src	TCP dst	Instructions
*	*	10.0.1.0/24	*	*	*	*	AA: Ouput Port 3
*	*	10.0.2.0/24	*	*	*	*	AA: Ouput Port 4

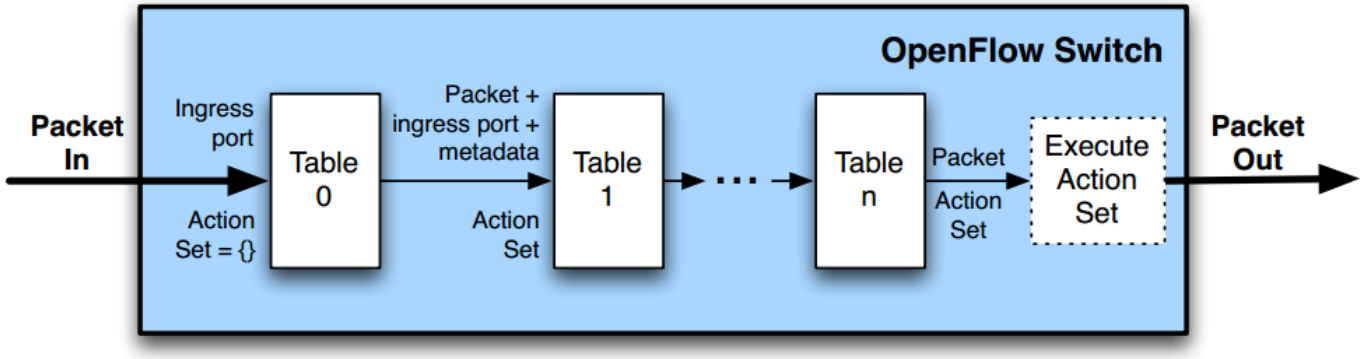
Firewall

MAC src	MAC dst	IP Src	IP Dst	Vlan	TCP src	TCP dst	Instructions
*	*	*	*	*	*	25	AA: Ouput Port 5
*	*	*	*	*	*	22	AA: Drop

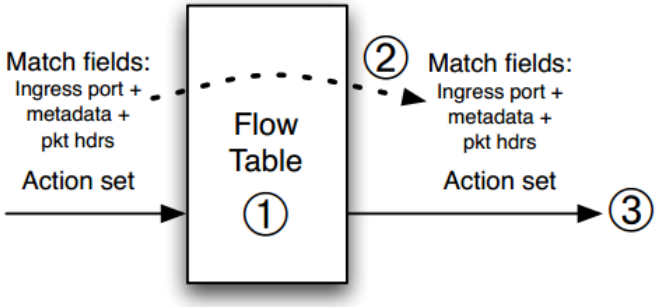
OpenFlow – Pipeline Process – 1

- OpenFlow pipeline contains multiple flow tables (at least one)
- OpenFlow pipeline processing define how packets interact with flow tables
- Metadata can be used to pass information between tables
- Action set: collection of actions attached to a packet. Action set is populated / updated by the instruction defined in the flow tables

OpenFlow – Pipeline Process – 1



(a) Packets are matched against multiple tables in the pipeline



① Find highest-priority matching flow entry

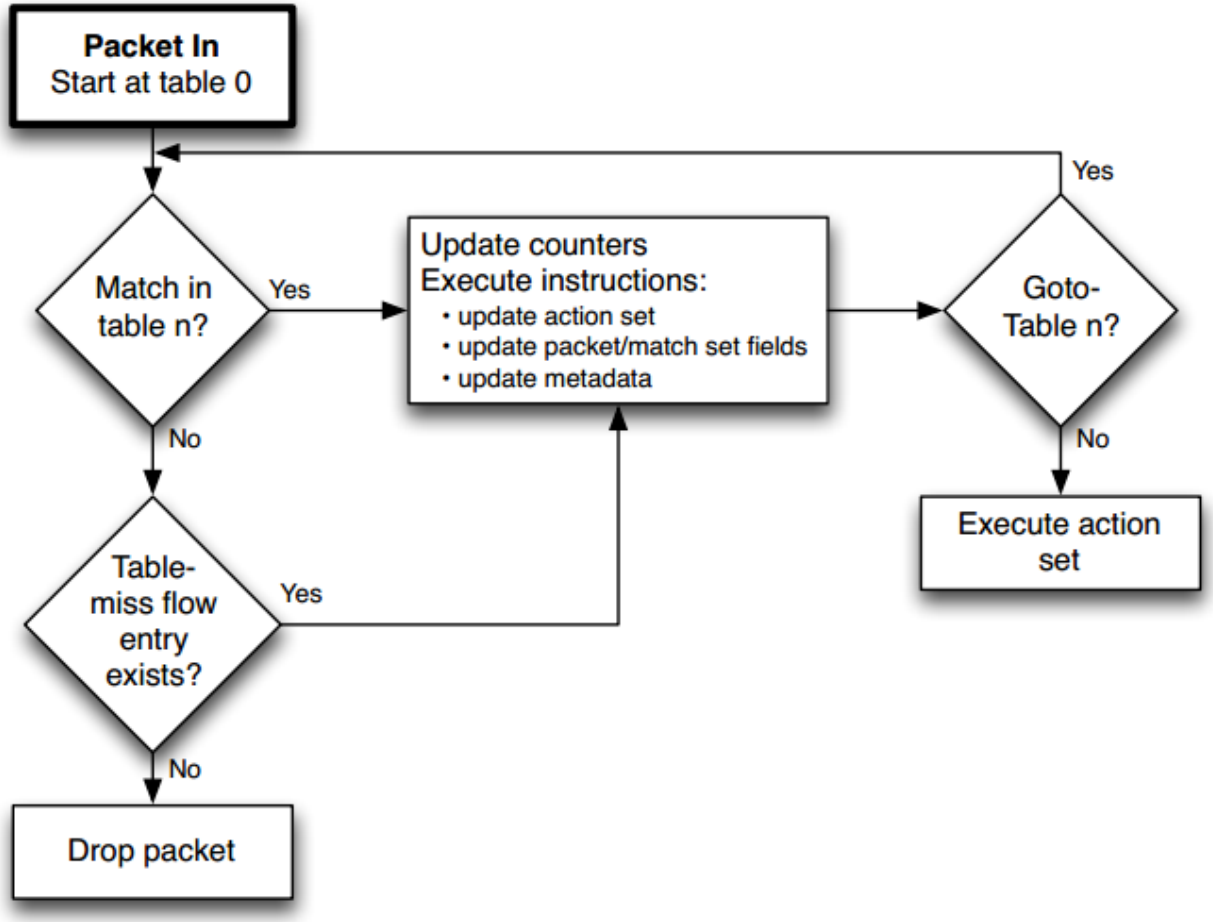
② Apply instructions:
- i. Modify packet & update match fields (apply actions instruction)
- ii. Update action set (clear actions and/or write actions instructions)
- iii. Update metadata

③ Send match data and action set to next table

(b) Per-table packet processing

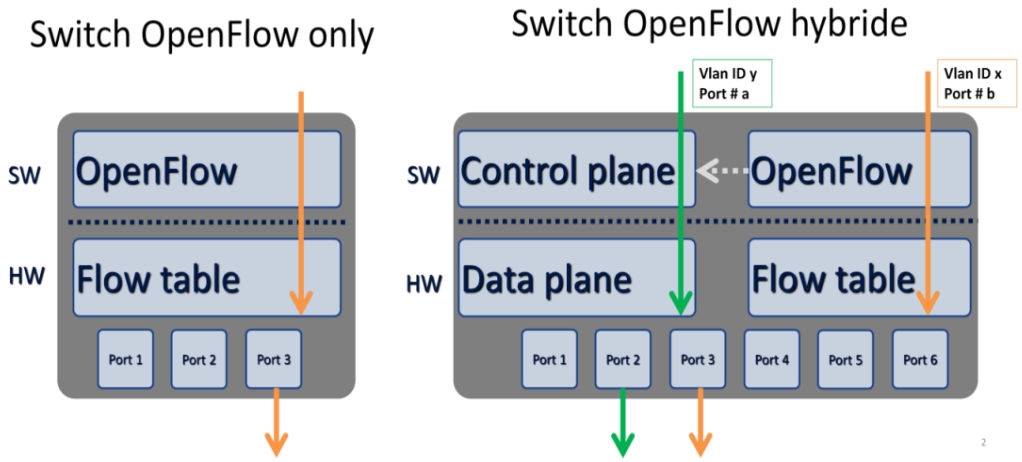
OpenFlow – Pipeline Process – 2

- Pipeline processing flowchart:



OpenFlow – Pipeline Process – 3

- 2 Different type of OpenFlow switch:
  - OpenFlow only: all packets processed by OpenFlow pipeline
  - OpenFlow hybrid: dual pipeline processing (openFlow and traditional)
- Classification mechanism (ex: VLAN tag) is used to redirect packets to different pipeline (OpenFlow or traditional)
- OpenFlow pipeline processing can pass packet to traditional pipeline



OpenFlow – Group table – 1

- A group table consists of group entries :

Group Identifier	Group Type	Counters	Action Buckets

  - group identifier: a 32 bit unsigned integer uniquely identifying the group
  - group type: to determine group semantics
  - counters: updated when packets are processed by a group
  - action buckets: an ordered list of action bucket, where each action bucket contains a set of actions to execute and associated parameters

- Flow entries can point to a group entry. Matching is still done by flow table
- Flow table allows forwarding to a specific port (or broadcast)
- Group Table enables OpenFlow to represent additional methods of forwarding (load-balancing, failover, multicast, flooding, etc.)

OpenFlow – Group table – 2

- Group Type is the key element which determines advance forwarding behavior :
  - **All** (required) : Execute all buckets in the group. This group is used for multicast or broadcast forwarding. The packet is cloned for each bucket.
  - **Indirect** (required) : This group supports only a single bucket. Allows multiple flow entries or groups to point to a common group identifier, supporting faster, more efficient convergence.
  - **Select** (optional) : Execute one bucket in the group based on a switch-computed selection algorithm (e.g. round robin). This group is used for load-balancing.
  - **Fast failover** (optional) : Execute the first live bucket (ordered). Each action bucket is associated with a specific port and/or group that controls its liveness. This group type enables the switch to change forwarding without requiring a round trip to the controller.

OpenFlow – Meter table – 1

- A meter table consists of meter entries :

Meter Identifier	Meter Bands	Counters

  - meter identifier: a 32 bit unsigned integer uniquely identifying the meter
  - meter bands: an unordered list of meter band, where each meter band specifies the rate of the band and the way to process the packet
  - counters: updated when packets are processed by a meter

- A meter measures and control the rate of packets assigned to a flow entry
- Meters enable OpenFlow to implement various simple QoS operations, such as rate-limiting
- Can be combined with queues (attached to ports) to implement complex QoS

OpenFlow – Meter table – 2

- A meter band consists of:

Band Type	Rate	Counters	Type specific arguments

  - band type: defines how packet are processed (drop or modify DSCP IP field)
  - rate: used by the meter to select the meter band, defines the lowest rate at which the band can apply (Kbps)
  - counters: updated when packets are processed by a meter band
  - type specific arguments: some band types have optional arguments

- Each meter may have one or more meter bands
- Packets are processed by a single meter band
- Meter applies the meter band with the highest configured rate that is lower than the current measured meter rate

OpenFlow – Counters

- Counters are maintained for each flow table, flow entry, port, queue, group, group bucket, meter and meter band
- Counters may be implemented in software and maintained by polling hardware counters with more limited ranges

OpenFlow – Counters

Counter	Bits	
Per Flow Table		
Reference Count (active entries)	32	*Required*
Packet Lookups	64	*Optional*
Packet Matches	64	*Optional*
Per Flow Entry		
Received Packets	64	*Optional*
Received Bytes	64	*Optional*
Duration (seconds)	32	*Required*
Duration (nanoseconds)	32	*Optional*
Per Port		
Received Packets	64	*Required*
Transmitted Packets	64	*Required*
Received Bytes	64	*Optional*
Transmitted Bytes	64	*Optional*
Receive Drops	64	*Optional*
Transmit Drops	64	*Optional*
Receive Errors	64	*Optional*
Transmit Errors	64	*Optional*
Receive Frame Alignment Errors	64	*Optional*
Receive Overrun Errors	64	*Optional*
Receive CRC Errors	64	*Optional*
Collisions	64	*Optional*
Duration (seconds)	32	*Required*
Duration (nanoseconds)	32	*Optional*
Per Queue		
Transmit Packets	64	*Required*
Transmit Bytes	64	*Optional*
Transmit Overrun Errors	64	*Optional*
Duration (seconds)	32	*Required*

OpenFlow – Instructions

- Each flow entry contains a set of instructions that are executed immediately when a packet matches the entry. These instructions result in changes to the packet, action set and/or pipeline processing.
  - **Meter** ***meter_id***: Direct packet to the specified meter.
  - **Apply-Actions** ***actions***: Applies the specific action(s) immediately.
  - **Clear-Actions**: Clears all the actions in the action set immediately.
  - **Write-Actions** ***actions:*** Merges the specified actions into the current action set.
  - **Write-Metadata** ***metadata*** : Writes the masked metadata value into the metadata field. (e.g new metadata = old metadata & ~mask j value & mask)
  - **Goto-Table** ***next-table-id***: Indicates the next table in the processing pipeline.
- The instruction set associated with a flow entry contains a maximum of one instruction of each type. The instructions of the set execute in the order specified by this above list.

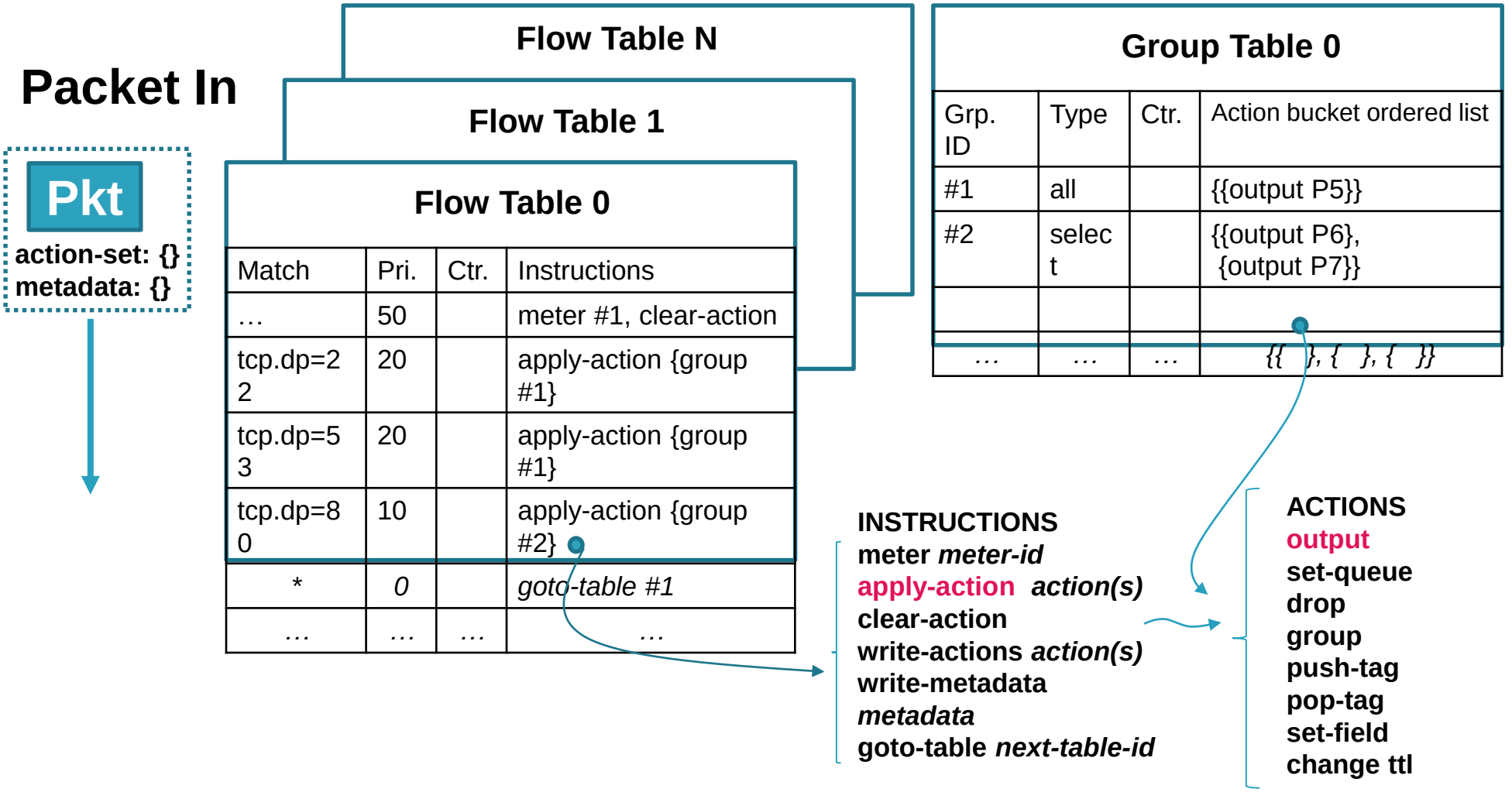
OpenFlow – Action

- Supported actions are :
  - **Output**. The Output action forwards a packet to a specified OpenFlow port
  - **Set-Queue**. The set-queue action sets the queue id for a packet
  - **Drop**. There is no explicit action to represent drops. Instead, packets whose action sets have no output actions should be dropped
  - **Group**. Process the packet through the specified group
  - **Push-Tag/Pop-Tag**. Switches may support the ability to push/pop tags (e.g:VLAN)
  - **Set-Field**. The various Set-Field actions are identified by their field type and modify the values of respective header fields in the packet
  - **Change-TTL**. The various Change-TTL actions modify the values of the IPv4 TTL, IPv6 Hop Limit or MPLS TTL in the packet
- Output can use special reserved ports like CONTROLLER, ALL, NORMAL

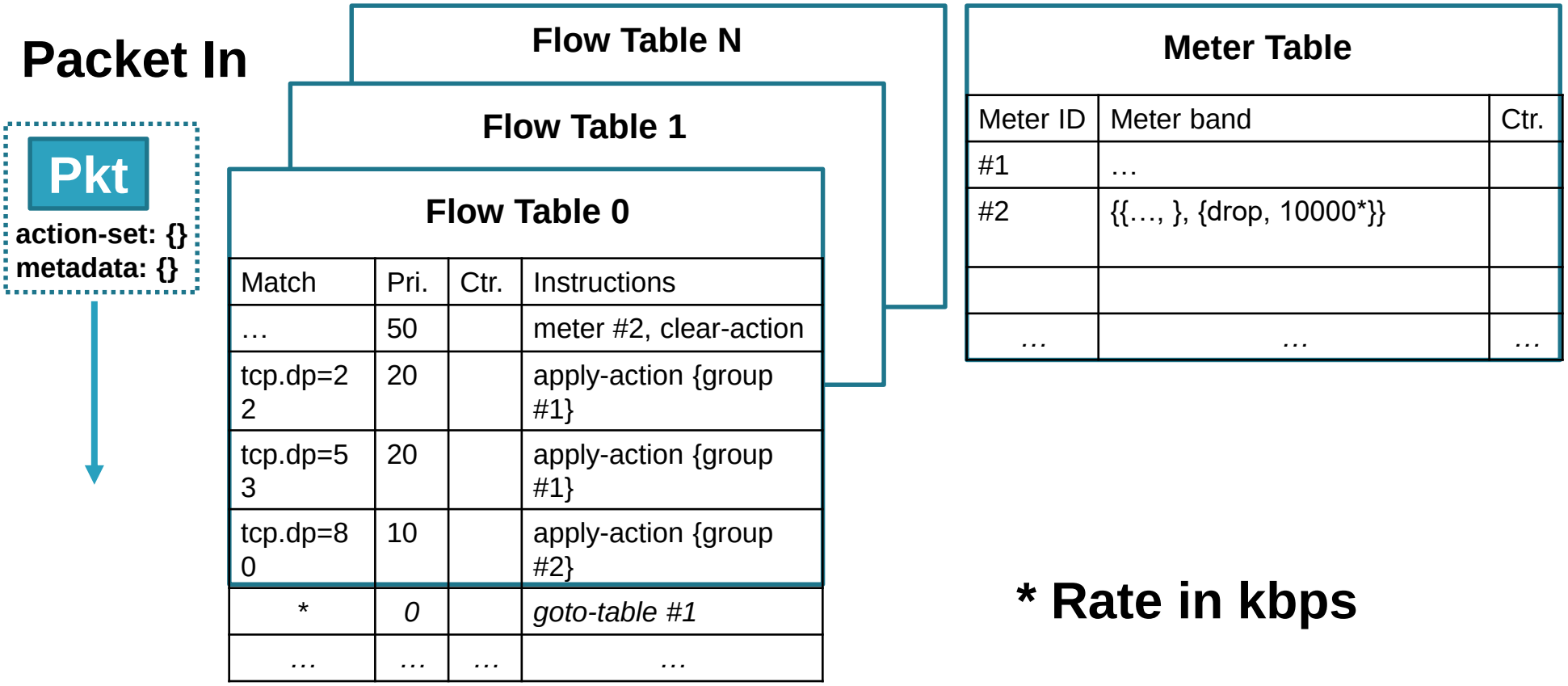
OpenFlow – Action Set – 1

- An action set is associated to each packet
- This set is empty by default
- A flow entry can modify the action set using a Write-Action instruction or a Clear-Action instruction associated with a particular match
- The action set is carried between flow tables
- An action set contains a maximum of one action of each type
- At the end of pipeline processing, actions in the action set of the packet are executed
- Actions in an Action Set are applied in a particular order

OpenFlow – Elements

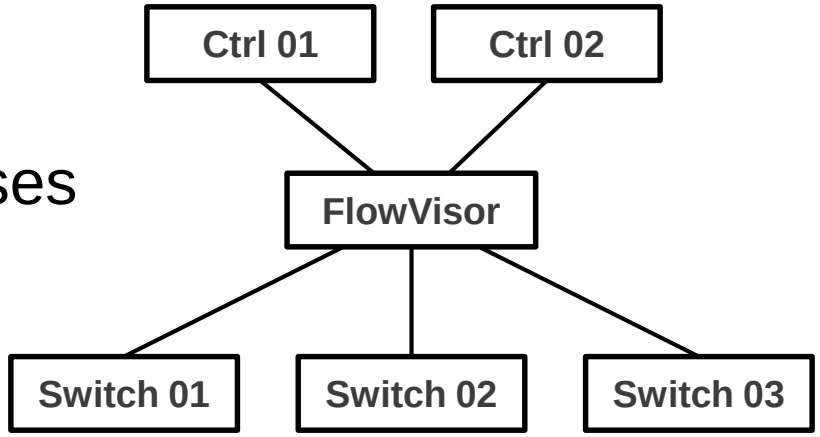


OpenFlow – Elements – 2



OpenFlow – Slicing

- FlowVisor allows to create slice of network resources and to control each slice with a different controller
- FlowVisor is a special purpose OpenFlow controller that acts as a transparent proxy between switches and controllers.
- Slices can be defined by:
  - Switch ports
  - Source / destination MAC addresses
  - Ethernet type
  - ICMP code / type
  - Source / Destination IP addresses
  - Source / Destination UDP/TCP ports



OpenFlow – Protocols

- Use TCP (port 6633), encryption possible with TLS
- Can use UDP with restricted functionalities
- When connection with controller is lost
  - Multiples controller (handover managed by controllers)
  - Else (or before first connection)
    - **Fail secure mode**: drop all packets addressed to controller
    - **Fail standalone mode**: act as a traditional switch by using manufacturer pipeline processing. Only hybrids switch can do that
- 3 types of messages
  - Controller to Switch
  - Asynchronous
  - Symmetric

OpenFlow – Protocols Msg – 1

- Controller/switch (messages initiated by the controller; may or may not require a response from the switch) :
  - **Features**: request the identity and the basic capabilities of a switch
  - **Configuration**: set and query configuration parameters in the switch
  - **Modify-State**: manage state on the switches (add, delete and modify flow/group entries in OpenFlow tables)
  - **Read-State**: collect various information from the switch (configuration, statistics and capabilities)
  - **Packet-out**: send packets out of a specified port on the switch. Packet contain an action set; an empty action list drops the packet.
  - **Barrier**: ensure message dependencies have been met or to receive notifications for completed operations
  - **Role-Request**: set/request the role of the controller OpenFlow channel
  - **Asynchronous-Configuration**: set an additional filter on the asynchronous messages that controller wants to receive

OpenFlow – Protocols Msg – 2

- Asynchronous (messages sent from the switch without a controller solicitation)
  - Packet-in: Transfer the control of a packet to the controller
  - Flow-Removed: Inform controller about the removal of a flow entry
  - Port-status: inform controller of a change on a port
  - Error: notify controllers of problems using error messages

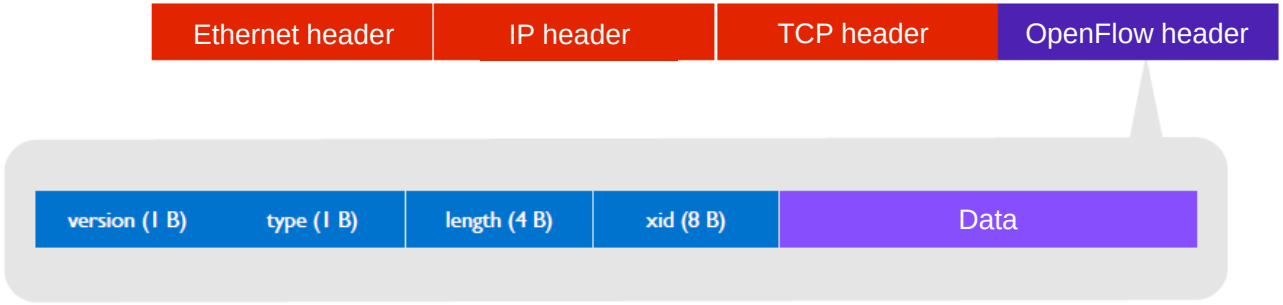
OpenFlow – Protocols Msg – 3

- Symmetric (messages sent without solicitation in either direction)
  - Hello: Hello messages are exchanged between the switch and controller upon connection startup.
  - Echo: Echo request can be sent from the switch or the controller, and must return an echo reply. They are used to verify the liveness and measure latency.
  - Experimenter: Experimenter messages provide a standard way for switches to offer additional functionality within the OpenFlow message type space.

OpenFlow - Flow instantiation

- They are 3 different mode which can be used to populate flow table:
  - Reactive flow instantiation
    - If no match for the flow is found, the switch creates an OFP packet-in packet and fires it off to the controller for instructions. Reactive mode reacts to traffic, consults the OpenFlow controller and creates a rule in the flow table based on the instruction
  - Proactive flow instantiation
    - Rather than reacting to a packet, an OpenFlow controller could populate the flow tables ahead of time for all traffic matches that could come into the switch.
  - Hybrid flow instantiation
    - A combination of both would allow flexibility of reactive for particular sets a granular traffic control that while still preserving low-latency forwarding for the rest of your traffic

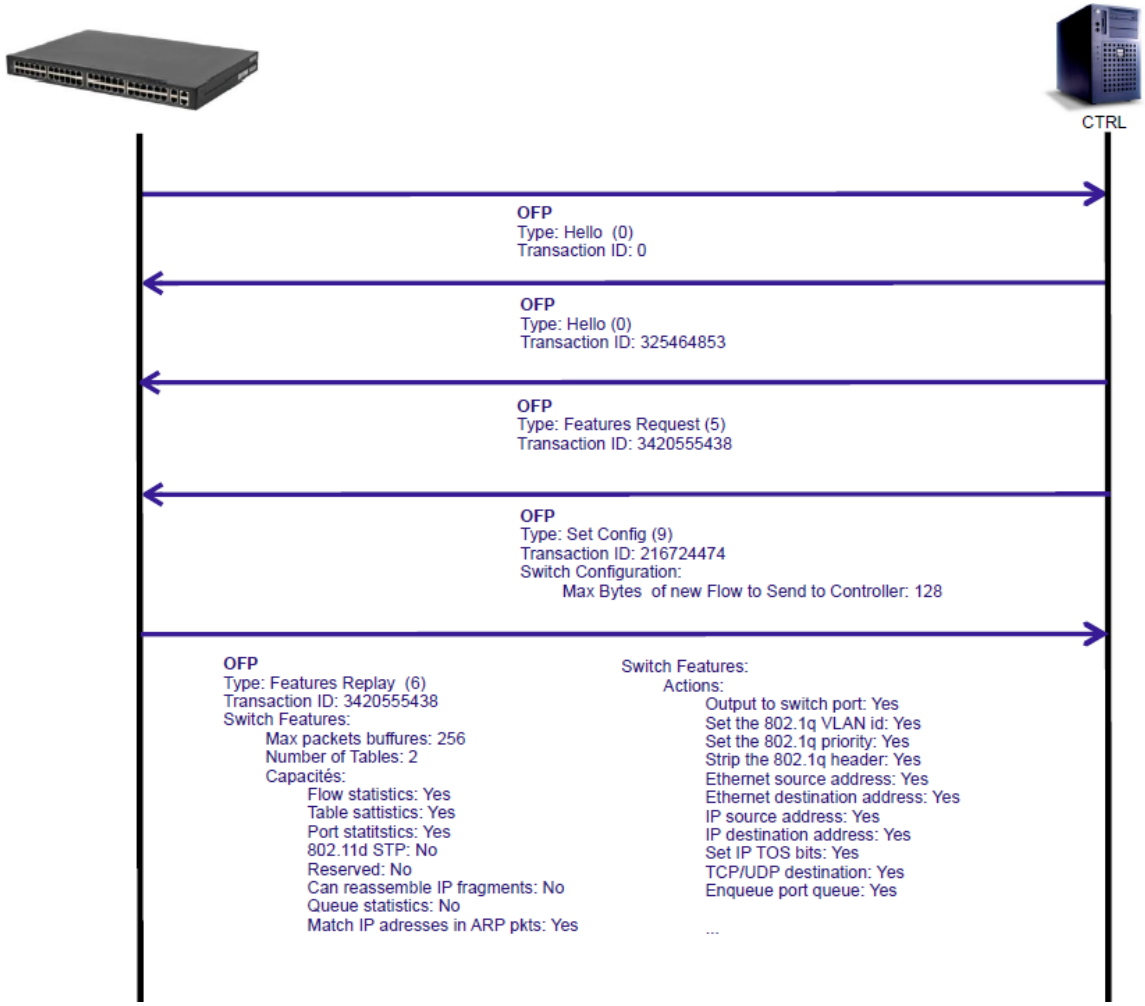
OpenFlow – Headers



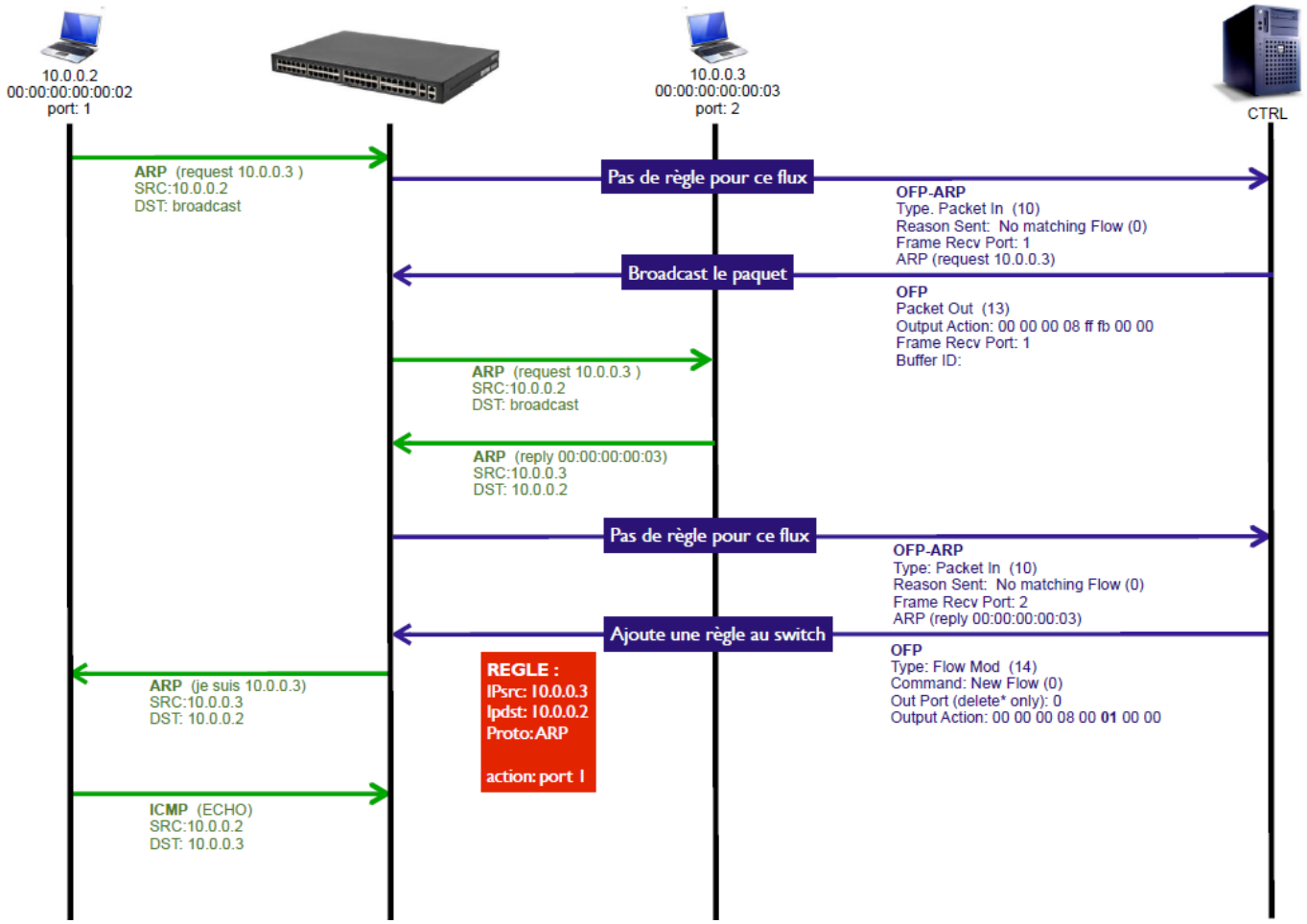
types
OFPT_HELLO
OFPT_ERROR
OFPT_ECHO_REQUEST
OFPT_ECHO_REPLY
OFPT_EXPERMINENTER
OFPT_FEATURES_REQUEST
OFPT_FEATURES_REPLY
OFPT_GET_CONFIG_REQUEST
OFPT_GET_CONFIG_REPLY
OFPT_SET_CONFIG

types
OFPT_PACKET_IN
OFPT_FLOW_REMOVED
OFPT_ECHO_PORT_STATUS
OFPT_PACKET_OUT
OFPT_FLOW_MOD
OFPT_GROUP_MOD
OFPT_PORT_MOD
OFPT_TABLE_MOD
OFPT_STATS_REQUEST
OFPT_STATS_REPLY

OpenFlow – Initialization

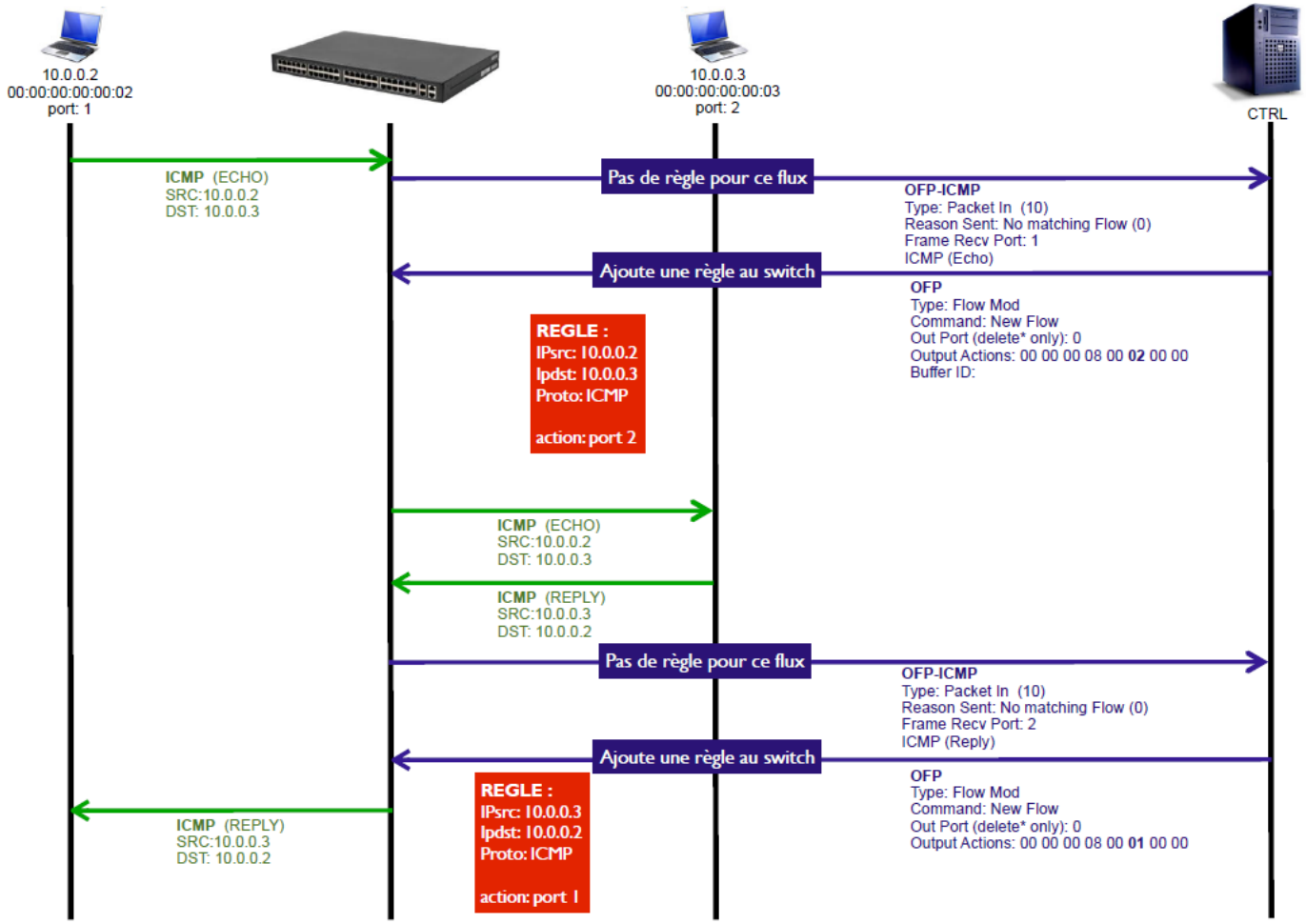


OpenFlow – Example – 1



Learning switch controller behaviors with ICMP

OpenFlow – Example – 2

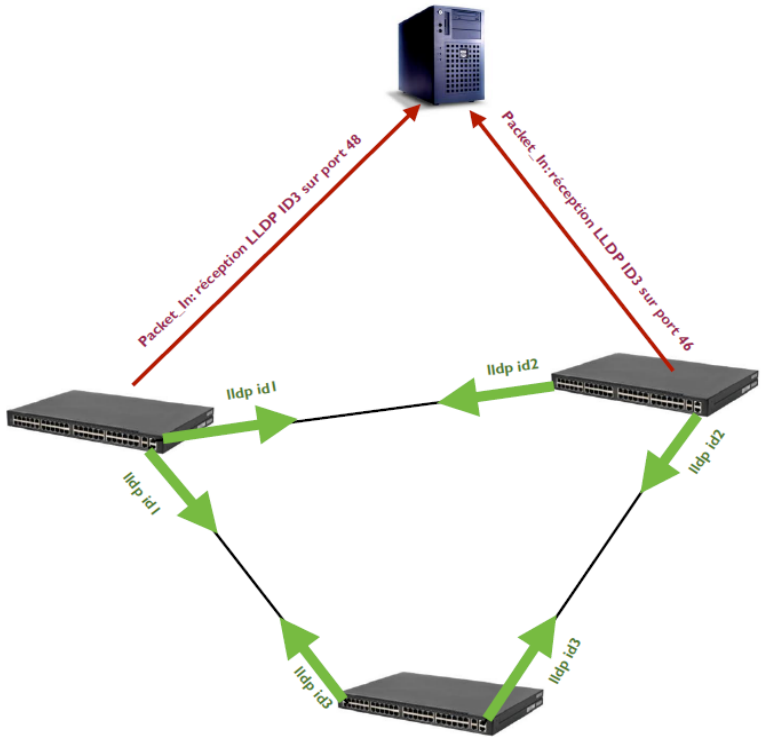


Learning switch controller behaviors with ICMP

OpenFlow – Network Discovery

- In an OpenFlow network, the controller performs network discovery
- This feature is currently in testing
- The Link Layer Discovery Protocol (LLDP) is used by network devices to advertise their identity, capabilities and neighbors
- LLDP frame has special multicast destination MAC and Ethertype (set to 0x88cc) identified by the network as an LLDP packet rather than a normal MAC frame

OpenFlow – Network Discovery



OpenFlow– Glossary

Glossary

References

http://networkstatic.net/openflow-proactive-vs-reactive-flows/

Lab simulation with Mininet

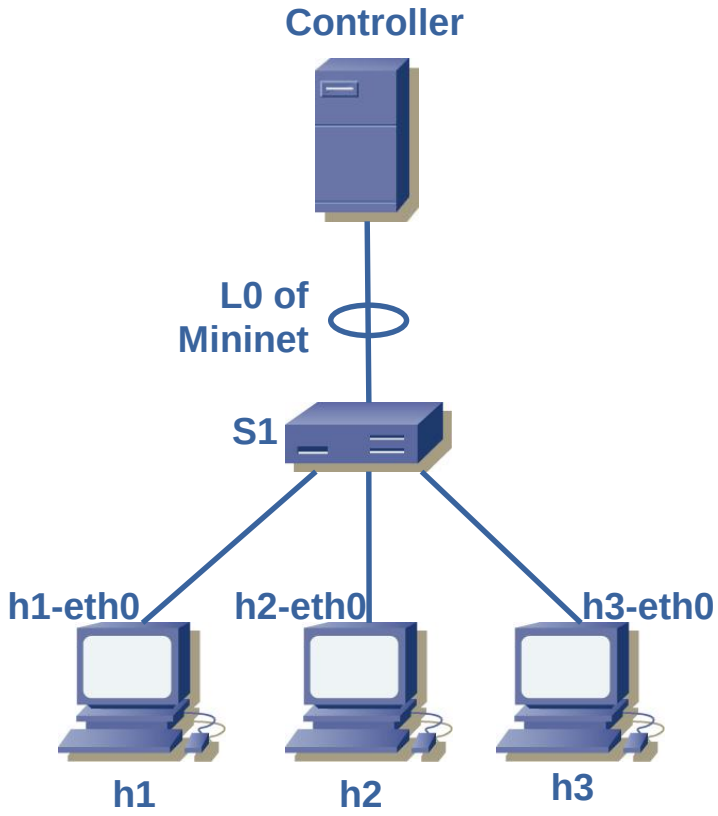
Lab Simulations (Mininet)

- Default topology with two hosts, one switch and one controller
  - Document and interpret with wireshark the information flow produced by the command " h1 ping h2"
  - Display analyze and interpret the flow table of S1
  - Determine the network behavior proposed by this topology (proactiv vs reactiv, bridging, routing, ….)

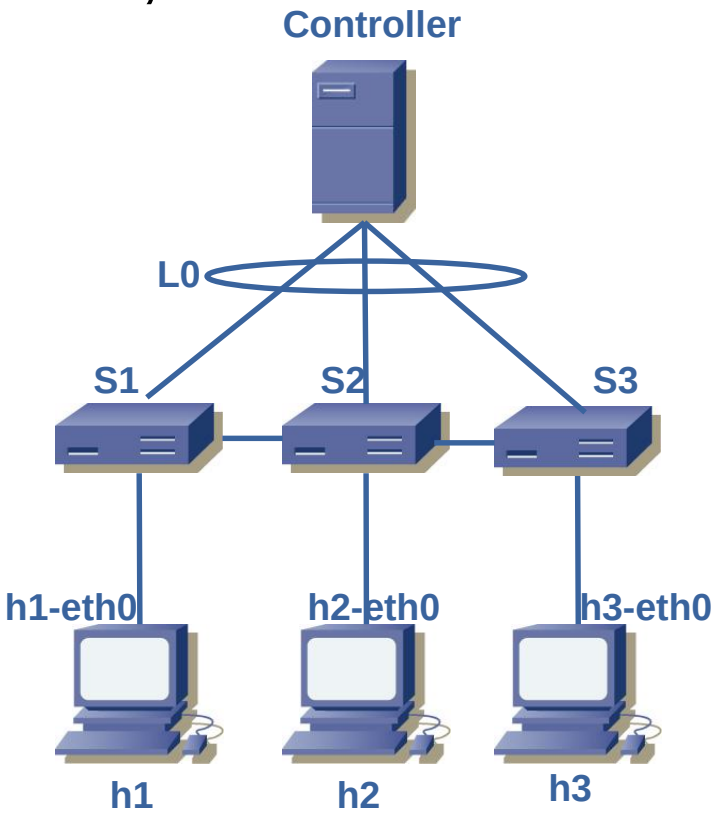
- Same exercise with a four host topology, connected to chained switches.
  - Document and interpret the information flow produced by the command " h1 ping h4" and "h4 ping h2"
  - Display the flow entry generated in the switch

Mininet topologies (--topo)

- Single,3



- Linear,3



Lab Simulations (Mininet)

- Run a simple web server on h1 and measure the traffic generated by getting the default web page from h3.
- Same exercise with a four host topology, connected to chained switches.
  - Document and interpret the information flow produced by the command " h1 ping h4" and "h4 ping h2"
  - Display the flow entry generated in the switch

- mininet> h1 python -m SimpleHTTPServer 80 &
- mininet> h2 wget -O - h1